SIH26080 — Regime-Aware Rainfall Post-Processing: Team Reference

2026-09-22

Overview

Raw weather forecasts (like GFS) have systematic errors that change depending on the type of monsoon day it is — an active spell, a break spell, a depression, or coastal/orographic rain each bias the forecast differently. Existing correction methods (quantile mapping, linear bias correction) apply one fixed fix regardless of the situation, so they under-perform exactly when it matters most: heavy rainfall days.

Our system does two things existing methods don't: (1) it identifies the monsoon regime first, from the forecast's own wind and pressure fields, then (2) applies a correction trained specifically for that regime, producing both a corrected rainfall number and a calibrated probability of crossing IMD's heavy-rainfall thresholds. The output is aggregated to district level — what a collector or disaster-management officer actually needs — and verified against real IMD observations using standard meteorological skill scores.

Everything in this doc is about a real, running prototype: real GFS/IMD data (2021–2025), a real trained classifier and correction models, real district-level output, and real verification numbers on a held-out 2025 test season. Nothing here is a mock-up.

Glossary

Everything below is used somewhere in this project. Grouped by topic so you can skip what you already know.

Meteorology terms

Term	Plain meaning
Regime	The overall type of monsoon weather happening right now: Active, Break, Depression, or Coastal/Orographic
Active spell	Monsoon trough over central India; widespread, heavy rain

Term	Plain meaning
Break spell	Trough shifts to the Himalayan foothills; central India goes dry
Depression	A low-pressure system (like a weak cyclone) that brings very heavy, concentrated rain along its track
Orographic rain	Rain caused by moist air forced upward by hills or mountains (e.g. Western Ghats)
GFS	Global Forecast System — NOAA's public weather model; our source of raw forecasts
IMD	India Meteorological Department — source of the real, observed rainfall we compare against
MSLP	Mean Sea-Level Pressure — used to detect depressions (a deep, closed low)
u850 / v850	Wind speed at 850 hPa (~1.5 km up), split into east-west (u) and north-south (v) components
NWP	Numerical Weather Prediction — the general term for models like GFS that simulate the atmosphere
24-h rainfall	Rain accumulated over 24 hours; IMD's day runs 03:00 UTC to 03:00 UTC the next day

IMD rainfall categories (the thresholds we predict against)

Category	Rainfall (24h)
Heavy	> 64.5 mm
Very heavy	> 115.5 mm
Extremely heavy	> 204.4 mm

Machine learning terms

Term	Plain meaning
CNN	Convolutional Neural Network — a network that learns patterns in grid/image-shaped data

Term	Plain meaning
U-Net	A CNN shape that shrinks an image down then rebuilds it back to full size, used here to output a corrected rainfall map the same size as the input
Tensor	A multi-dimensional array of numbers — our input is a 6×129×135 tensor: 6 channels, on a 129×135 grid
Channel	One layer of the input tensor (e.g. rainfall, wind, elevation are each one channel)
z-score	Rescaling a number to (value − mean) / spread, so every channel is on a comparable scale before training
log1p	log(1 + x) — applied to rainfall before z-scoring, because rainfall values are extremely skewed (mostly near 0, rarely 300mm)
Softmax	Turns raw network outputs into probabilities that sum to 1 (used by the regime classifier)
Sigmoid	Squashes a number to between 0 and 1 (used for each exceedance probability independently)
Loss function	The formula the network is trained to minimise; ours is deliberately asymmetric (see Technical deep-dive)
Epoch	One full pass through the training data
Class-weighted	Rare categories (e.g. Depression days) count for more in the loss, so the model doesn't ignore them
Overfitting	When a model memorises training data instead of learning general patterns — a real risk here since we only have ~250 training days
Frozen (weights)	Once training ends, the model's numbers don't change any more — the live system just runs the frozen model, it doesn't keep learning

Verification / skill-score terms

Term	Plain meaning
RMSE	Root Mean Squared Error — average size of the forecast error, in mm
POD	Probability Of Detection — of the heavy-rain events that happened, what fraction did we catch?
FAR	False Alarm Ratio — of the times we said "heavy rain", what fraction didn't happen?
CSI	Critical Success Index — balances POD and FAR into one score (higher is better)
ETS	Equitable Threat Score — like CSI, but corrected for hits you'd get by chance
FSS	Fractions Skill Score — checks if heavy rain was forecast in roughly the right *area*, not the exact cell
Brier score	How well-calibrated a probability is (0 = perfect, 0.25 = a coin flip)
Contingency table	The hits / misses / false-alarms / correct-negatives count that POD, FAR, CSI, ETS are all built from

Data / engineering terms

Term	Plain meaning
GRIB	The file format weather models store forecasts in
Shapefile	A file format storing map polygons (here: the outline of every district)
DEM	Digital Elevation Model — a map of ground height
Grid cell	One square of our 0.25° × 0.25° (~27 km) forecast grid
GADM	A free global database of administrative boundaries (district outlines); used as a placeholder for the official Survey of India shapefile

Data sources

Source	What it is	Why this one
GFS 0.25° (NOAA public archive, via `herbie`)	Raw forecast: rainfall + u850/v850 wind + MSLP, issued daily at 00Z	The thing we correct. Free, public, same variables NCMRWF's own model provides — swapping to NCMRWF later is a config change, not a rewrite
IMD 0.25° daily gridded rainfall (via `imdlib`)	What actually fell, same day, same grid	Ground truth: both the training target and the answer key for grading. IMD's own official product
SRTM 90m elevation (DEM)	Static map of ground height	Rain behaves differently in hills (forced uplift) vs plains — the model needs to know where the hills are
District boundary shapefile (GADM)	Polygon outline of all ~676 districts	Turns a grid of numbers into something a district officer can read (see District aggregation)

Five monsoon seasons (JJAS, 2021–2025) were downloaded for GFS and IMD. The prototype trains on 2023–2024 and tests purely on 2025 — an entire season the model has never seen, never individual random days (see Technical deep-dive for why that split matters).

Pipeline walkthrough

Two separate pipelines: one runs once offline to train the models (pre-training), one runs every single day operationally (system workflow). Full illustrated versions of both were shared in chat and live in `docs/pretraining_pipeline.svg` and `docs/system_workflow.svg` in the repo — this is the fast-read version.

Pre-training (offline, once per season)

```
flowchart TD
    A[GFS + IMD + DEM<br/>5 seasons of data] --> B[Align to one
grid<br/>129x135, z-scored]
    B --> C[Unified tensor<br/>6 channels per day]
    A --> D[Label each day's regime<br/>from OBSERVED rain]
    C --> E[Stage A: train classifier<br/>input: wind+pressure only]
    D --> E
    C --> F[Stage B: train 4 U-Nets<br/>one per regime]
    D --> F
    F --> G[Frozen classifier + 4 U-Nets<br/>saved for daily use]
    E --> G
```

System workflow (operational, every forecast day)

```
flowchart TD
    A[Today's 00Z GFS run] --> B[Align + normalise<br/>same code as training]
    B --> C[Regime classifier<br/>frozen weights]
    C --> D[Router picks 1 of 4<br/>trained U-Nets]
    D --> E[Corrected rain +<br/>3 exceedance probabilities]
    E --> F[District aggregation<br/>676 districts]
    F --> G[Dashboard:<br/>map, table, alerts]
    H[IMD observations<br/>next day] --> I[Verification:<br/>RMSE, POD, FAR,
CSI, ETS, FSS]
    E --> I
    I --> G
```

The stages in words

1. **Ingest**: pull today's GFS forecast (rainfall, wind, pressure).
2. **Align**: put it on the same 129×135 grid as IMD, using the *saved* training statistics for normalisation — not recalculated, so the live system behaves identically to what was trained.
3. **Classify**: the frozen classifier reads wind+pressure only (never rainfall — this is what lets it run on a forecast where rainfall hasn't happened yet) and outputs a regime.
4. **Correct**: the one U-Net matching that regime runs, outputting corrected rainfall (mm) plus P(>64.5mm), P(>115.5mm), P(>204.4mm).
5. **Aggregate**: the grid is converted to one row per district (mean = advisory value, max = flood-risk flag).
6. **Publish**: the dashboard shows raw vs corrected vs observed, the district table, and the regime for that day.

7. **Verify** (next day, once IMD's real observation is available): raw vs corrected are both scored against what actually happened, and that score feeds the season-long skill report.

Technical deep-dive

Regime labels: how a day gets tagged

No one hand-labels days. Labels come automatically from *observed* rainfall and pressure (Rajeevan, Gadgil & Bhate, 2010):

1. Average observed rainfall over a fixed "core monsoon zone" (18–28°N, 65–88°E), every day, for 32 years (1991–2022 climatology).
2. For each calendar day-of-year, compute the long-term mean and spread — the "normal" for that date.
3. More than 1 standard deviation above normal for 3+ consecutive days → **Active**. Below normal, 3+ days → **Break**.
4. A closed low in the forecast's own pressure field (≥4 hPa deeper than its surroundings, below 1000 hPa) in the Bay of Bengal/trough region → **Depression** (overrides Active/Break).
5. Everything else → **Coastal/Orographic**.

This is fully reproducible from public data — no manual judgment calls. Known limitation: the MSLP depression threshold is a simplified proxy for IMD's official definition (which also needs sustained wind speed and a tracked system), so it currently over-flags some borderline days. It's a tunable config value, not a hidden assumption.

Model architectures

Stage A — Regime classifier: a small CNN (~106,000 parameters, deliberately small since we only have ~250 training days). Input: wind (u850, v850) + MSLP only — never rainfall. Four `[Conv3x3 → BatchNorm → ReLU → MaxPool]` blocks shrink the image while extracting synoptic-scale patterns, then a final layer outputs 4 probabilities (softmax) summing to 1. Trained with class-weighted cross-entropy so rare regimes (e.g. Depression) aren't ignored.

Stage B — Correction: two variants are trained; whichever scores better on validation is auto-selected as the one used live.

Variant	Description
4 regime-specific U-Nets	Each sees only its own regime's training days. Depression uses 7x7 convolution kernels (wider receptive field, matching how depressions are large synoptic systems); Break uses extra L2 weight decay (discourages inventing rain on dry days)
1 shared U-Net	Trained on all days regardless of regime, no specialisation

Every U-Net outputs 4 numbers per grid cell: corrected rainfall (mm, via softplus so it's never negative) and 3 probabilities — P(>64.5mm), P(>115.5mm), P(>204.4mm), via sigmoid, kept monotonic (each can't exceed the one before it).

The loss function (why it's asymmetric)

Plain MSE (mean squared error) treats over-predicting and under-predicting as equally bad. Ours doesn't: an *extra* penalty is added specifically when the model under-predicts a cell that actually saw heavy rain (>64.5mm). Over-predicting is not penalised extra. Reasoning: missing a genuine heavy-rain warning is a worse real-world outcome than issuing one that turns out mild — the loss is shaped to match that asymmetry deliberately, not just to minimise average error.

Why the train/test split matters

Train on 2023–2024, test on 2025 — *entire seasons*, never random individual days. Two days three days apart in the same season are highly correlated (same weather pattern); a random split would leak information from days next to the test days into training and produce falsely optimistic scores. A season the model has genuinely never seen is the only honest test.

District aggregation deep-dive

Why it's needed at all

Every model output is a grid of 129×135 numbers. Nobody governs by grid cell — a district collector, NDRF team, or IMD bulletin operates in terms of districts, whose boundaries don't line up with a 27 km square grid. A single grid cell often straddles 2–4 districts; a

large district can contain dozens of cells. A shortcut like "use the value of the nearest cell to the district's centre" silently gets coastal and border districts wrong, because part of the district might sit in a rainy cell and part in a dry one.

What it does, step by step

1. **Load the shapefile** — the exact polygon outline of every district (GADM for the prototype; swap in the official Survey of India boundaries before any real deployment, since GADM's depiction of India's external border differs from the official map).
2. **Turn every grid cell into its own tiny polygon** — a 0.25°×0.25° square, one per cell.
3. **Overlay the two polygon sets**, in an *equal-area* map projection (so "area" means real km², not degrees, which distort differently near the equator vs the poles). This finds exactly how many km² of each district falls inside each cell.
4. **Turn that into a weight**: `weight = (overlapping area) / (district's total area)`. Every district's weights sum to exactly 1 across the cells that touch it.
5. **Cache it as a sparse matrix** (676 districts × 17,415 cells) — computed once, ever, since boundaries and grid cells are both fixed.
6. **Aggregating any day is then one matrix multiply**: `district_values = weight_matrix @ grid_values`. Milliseconds, no geometry recomputed daily.

Two aggregations, two different jobs

Aggregation	Used for	Why
Area-weighted mean	Advisory value	Represents the district as a whole — what a farmer or general bulletin should see
Area-weighted maximum	Flood-risk flag	A district can have a modest average but one small pocket of extreme rain (e.g. a hill sub-catchment) that actually triggers a flash flood — averaging would hide it, taking the max catches it

The exceedance probabilities are aggregated the same way (by max), and that district-level max probability sets the alert colour (Yellow/Orange/Red).

What's built vs designed-but-not-built

So nobody on the team overclaims in front of judges.

Built and running

- Full data pipeline: GFS/IMD/DEM/district download, grid alignment, tensor + normalisation (`src/rainfall/data/`)
- Regime labelling from observations (`src/rainfall/regimes/labels.py`)
- Regime classifier, trained (`classifier.pt`)
- Both U-Net variants (4 experts + 1 shared), all trained and saved (`unet_*.pt`)
- District aggregation engine with a cached weight matrix (`src/rainfall/aggregate/districts.py`)
- Verification module: RMSE, ETS, CSI, POD, FAR (via `xskillscore`) + FSS (hand-implemented), all at 3 thresholds
- Live Streamlit dashboard: per-day maps, district choropleth with alerts, season-wide verification table, regime timeline, case gallery, and a "run today's real forecast" button
- 366 real forecast-days processed end-to-end — zero synthetic or placeholder data anywhere

Designed, not yet built

- [ ] Day-2/Day-3 forecast leads (config already supports it, not yet run — time, not difficulty)
- [ ] Cartosat 30m DEM swap-in (currently SRTM 90m — functionally near-identical at this grid resolution)
- [ ] Official Survey of India district boundaries (currently GADM)
- [ ] Western disturbances as a regime (needs Oct–May data + a 500 hPa channel — outside JJAS scope)
- [ ] Hosted/scheduled deployment (the operational script works locally; not running on a server anywhere)

Worked example: 19 June 2025

A real test-season day — every number below came out of the actual pipeline, nothing mocked.

1. Ingest. The 00Z GFS run from 18 June 2025 is fetched. Its Day-1 forecast (valid 19 June) gives rainfall + wind + pressure on the 129×135 grid.

2. Build the tensor. Combined with the cached elevation/land-mask channels into one 6×129×135 block, normalised with the saved training statistics.

3. Classify. The classifier outputs: Coastal/Orographic 0.54, Depression 0.26, Active 0.19, Break 0.004 — picks **Coastal/Orographic**. Worth flagging: the label derived from what *actually* happened that day was **Active** — a genuine classifier miss, useful for an honest "how confident is your classifier" answer.

4. Correct. The U-Net runs, producing a corrected rainfall map and three probability maps.

5. Verify against what actually happened:

	Raw GFS	Corrected (shared U-Net)
RMSE (mm)	21.6	27.7
POD (heavy rain caught)	0.32	0.85
FAR (false alarms)	0.50	0.66
CSI	0.226	0.293
ETS	—	0.29
Hits / Misses / False alarms	64 / 138 / 63	172 / 30 / 336

202 grid cells observed >64.5mm that day, max 271mm. Raw caught only 64 of them (missed 138). Corrected caught 172 of the same 202 (missed only 30), at the cost of more false alarms and a worse average error — the asymmetric loss working exactly as designed: it traded precision for recall on heavy rain, because that's the trade we told it to make. CSI (which balances both) still nets positive.

Honest weakness to flag here too: the corrected *amount* is currently biased high (predicted a district mean of ~234mm where ~110mm actually fell). The probability output is well-calibrated (Brier score 0.041 at the heavy threshold, where 0 is perfect and 0.25 is a coin-flip) — the raw millimetre number needs the loss's λ weight tuned down, or more training epochs, before it's presentation-ready as a stand-alone number.

6. District aggregation — real output, top of the list:

District	State	Raw max (mm)	Corrected max (mm)	Observed max (mm)	P(>64.5mm)	Alert
Thane	Maharashtra	219	307	201	0.99	Red
Palghar	Maharashtra	219	307	191	0.99	Red
Mumbai Suburban	Maharashtra	117	307	201	0.99	Red
Mumbai City	Maharashtra	55	271	201	0.98	Red

Mumbai City is the clean example: raw forecast said 55mm (nowhere near heavy); corrected and the actual observation (201mm) both say otherwise. The correction moved the forecast in the right direction, even though the exact magnitude overshot.

7. Dashboard. All of the above — three maps, the regime badge with its honest confidence numbers, the metrics table, the district choropleth, the flagged-district list — is what actually renders in the Streamlit app for this date, live, reading only pre-computed files.

8. The loop closes. Tomorrow's forecast runs through the same steps, and this day becomes one more row in the season-long verification table judging the whole system.

Current results and honest limitations

Season-wide skill (122 held-out test days, 2025, Day-1 lead)

Metric (@ >64.5mm)	Raw GFS	Corrected (shared U-Net)
CSI	0.091	0.137
Brier score (>64.5mm probability)	—	0.041
Brier score (>115.5mm probability)	—	0.015
Brier score (>204.4mm probability)	—	0.004

The shared U-Net currently edges out the 4 regime-specific experts on this particular run (CSI 0.137 vs 0.125) — the pipeline auto-selects whichever scores better, so this isn't a fixed claim, it's measured each time the models retrain. Both comfortably beat raw GFS.

Known limitations (say these before a judge finds them)

1. **Corrected rainfall amount is currently biased high.** POD improves sharply (more heavy-rain events caught) but at the cost of over-predicting magnitude and more false alarms. The probability outputs are well-calibrated even though the raw mm number isn't yet — needs loss-weight tuning or more epochs.
2. **Only 2 training seasons used** (2023–2024) for this run, out of 5 downloaded. More training data is available and not yet used to save time before the deadline.
3. **Depression label over-flags slightly** — the MSLP proxy threshold is looser than IMD's official depression definition.
4. **Classifier accuracy is modest** on the rarer regimes (Active, Depression) given only ~250 training days across 4 imbalanced classes — it leans toward predicting the majority class (Coastal/Orographic). The confusion matrix is in the dashboard, not hidden.
5. **District boundaries are GADM**, not the official Survey of India shapefile — fine for a prototype, must be swapped before any real deployment claim.

Repo map

```
sih26080/
├─ configs/config.yaml         years, leads, thresholds, regime parameters — change behaviour here, not in code
├─ src/rainfall/
|  ├─ grid.py                  the master 129x135 grid definition
|  ├─ data/                    gfs.py, imd.py, dem.py, districts.py, build_dataset.py
|  ├─ regimes/                 labels.py (Rajeevan index, MSLP), classifier.py (the CNN)
|  ├─ correction/              unet.py (U-Net, asymmetric loss, 4 experts)
|  ├─ aggregate/               districts.py (area-weighted mean/max, alerts)
|  └─ verify/                  metrics.py (xskillscore + FSS)
├─ scripts/                    01_download → 02_build_dataset → 03_labels → 04_train → 05_precompute → 06_predict_today → 07_figures, run in that order
├─ dashboard/app.py            the Streamlit app
├─ docs/                       flowcharts (SVG) + generator scripts, this doc's source material
└─ data/processed/             trained models (`models/`), predictions, precomputed tables the dashboard reads
```


To run everything from scratch: `README.md` in the repo root has the exact command sequence. Most scripts are resumable/cached — re-running `01_download.py` after an interruption picks up where it left off rather than re-downloading everything.